RAMDEOBABA UNIVERSITY, NAGPUR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B.Tech. CSE (Artificial Intelligence and Data Science) (AIDS)

SEMESTER V

SI LAB

PROJECT REPORT

STATISTICAL ANALYSIS AND INFERENTIAL MODELING OF GLOBAL LISTED COMPANIES' MARKET CAPITALIZATION DATA

Submitted by

Student Name	Roll No.	Enrollment No.
Aayush Khiratkar	25	24RBU/UG/CS/02/000006

Section: B1-B2

Academic Year: 2026–27

Statistical Analysis and Inferential Modeling of Global Listed Companies' Market Capitalization Data

1. PROJECT OBJECTIVE

The purpose of this project is to find out what is related to the market capitalization of the world's largest listed companies: the size of the company (revenue and number of employees), its country or region, or its sector. Live company data is collected from Yahoo Finance, cleaned and converted to US dollars, and then studied with descriptive statistics, probability distributions, sampling, hypothesis tests, regression and classification models.

The expected output is a cleaned dataset, a set of statistical test results, a comparison of several regression and classification models against a "no skill" baseline, a saved best model, and a prediction interface. The classification task asks whether a company belongs to the **High market-cap tier** (the top third of companies by market cap) using only its revenue, employees, region and sector. The project is intended for academic study in the SI Lab and for building understanding of statistical inference and model evaluation. It is not intended as investment advice.

The objectives of the project are:

1. Collect live company data from the Yahoo Finance API, convert it to USD and check its quality.
2. Describe the data and test which probability distributions fit it.
3. Compare sampling techniques and demonstrate the Central Limit Theorem.
4. Test hypotheses (t-tests, ANOVA, chi-square, correlation) about size, region and sector.
5. Preprocess the data and prepare it for model building, using the log scale for skewed variables.
6. Train multiple classification models (Logistic Regression, Decision Tree, Random Forest, KNN and SVM) and a majority-class baseline.
7. Evaluate and compare the models using accuracy, precision, recall, F1-score and confusion matrices, with a majority-class baseline as the yardstick.
8. Select the best model, save it, and use it in a prediction interface that accepts input and displays predictions.

2. INTRODUCTION

Market capitalization (market cap) is the total market value of a company's shares, calculated as share price multiplied by the number of shares outstanding. It is the most common way to rank companies by size and is closely watched by investors, analysts and regulators. A natural question is what a company's market value is actually related to. Is it simply the size of its business, measured by sales and workforce, or do the country where it is listed and the sector it operates in also matter?

This project answers that question with data. A sample of large listed companies from the United States, Europe, Canada, Asia, Latin America, Australia and Israel is pulled from Yahoo Finance. Market cap and revenue are both extremely right-skewed: a handful of giant companies dominate the totals. This makes the choice of statistical method important, and the project therefore works on the log10 scale wherever the data requires it and checks the assumptions behind every test before using it.

2.1 Problem Statement

Given a company's revenue, number of employees, region (US or Non-US) and sector, predict whether it falls in the High market-cap tier, and statistically examine whether company size, region and sector are related to market capitalization. The project also reports honestly whether the models perform better than a baseline that simply predicts the majority class.

2.2 Scope of the Project

The project includes the following functions:

- Automated data collection from the Yahoo Finance API (via the yfinance library) with a raw-data backup and currency conversion to USD.
- Data quality checks, feature engineering and exploratory data analysis.
- Descriptive statistics, probability distribution fitting, sampling techniques and the Central Limit Theorem.
- Hypothesis testing: one-sample, two-sample and paired t-tests, one-way and two-way ANOVA, chi-square test and correlation tests (Pearson and Spearman).
- Regression models (linear, Ridge, Lasso, Decision Tree, Random Forest) and classification models, each compared with a baseline on a held-out test set.
- Saving the best models and a prediction interface.

The limitations of the project are as follows. The companies are hand-picked large companies that exist today, mostly US-listed or ADRs, so the findings describe this sample and not all listed companies. Market cap is a single snapshot and revenue is trailing twelve months in the reporting currency, both converted at one recent exchange rate. The models see only four inputs, whereas market value mostly reflects profit, growth and risk, which are not included. Several groups are small, so some tests have limited power.

3. PROJECT METHODOLOGY

The project followed these stages: data collection, data quality checks and feature engineering, exploratory analysis, preprocessing, feature selection, training of multiple models, evaluation and comparison, selection and saving of the best model, and deployment through a prediction interface. A single random seed (42) and a 5% significance level are used throughout, so every run of the notebook gives the same numbers.

3.1 Dataset Loading and Exploration

Dataset name and source: Global Listed Companies Market Capitalization Dataset, collected by the student from Yahoo Finance using the yfinance Python library

(https://pypi.org/project/yfinance/). The data was pulled on 29 October 2026; the raw pull is backed up in market_cap_raw.csv and the cleaned dataset is saved as market_cap_data.csv.

A list of 123 tickers was requested, covering the United States, Taiwan, Japan, Korea, Europe, Canada, China, India (ADRs), Latin America, Australia and Israel. 122 companies had a usable market cap and revenue and form the final dataset (from 20 countries and 9 sectors), with 122 rows and 18 columns after feature engineering (11 columns in the raw pull). The target variable for classification is High_Cap (1 = High tier, 0 = Low or Medium tier). Table 1 lists the variables.

Table 1: Variables in the dataset

Variable	Data type	Description
Ticker, Company	Text	Ticker symbol and company name from Yahoo Finance
Country, Sector	Categorical	Country of headquarters and business sector
MarketCap_B	Numeric	Market capitalization in USD billion (converted from the trading currency)
Revenue_B	Numeric	Total revenue in USD billion (converted from the reporting currency)
Employees	Numeric	Full-time employees (missing for some companies)
Region	Categorical	US or Non-US
PS_Ratio	Numeric	Price-to-sales ratio = market cap / revenue
log_MarketCap, log_Revenue, log_Employees	Numeric	log10 versions, because the data is heavily right-skewed
Cap_Tier	Categorical	Low / Medium / High, equal thirds by market cap
Sector_Group	Categorical	Sector name if it has at least 5 companies, otherwise “Other”
High_Cap	Binary (target)	1 if Cap_Tier is High, otherwise 0

The exploration step displayed the first rows of the data (df.head), the data types and non-null counts (df.info), the shape, summary statistics (df.describe), missing values per column, duplicate tickers, and the largest revenues as a sanity check on the currency conversion. Exploratory visualizations include the number of companies by country, histograms of market cap and revenue, a revenue versus market cap scatter plot, a correlation heatmap, and a box plot of market cap by sector. Because Cap_Tier is built from equal thirds of market cap, the High class holds about one third of the companies by construction; the exact class counts are High = 41, Others = 81.

3.2 Data Preprocessing

1. **Currency conversion:** market cap (trading currency) and revenue (reporting currency) were converted to USD using the latest exchange rates from Yahoo Finance, with a documented fallback table used only if the live lookup failed.
2. **Missing values and duplicates:** companies with no market cap or revenue were skipped at collection time (122 of the 123 requested tickers were kept). No duplicate

tickers were found. Employees is missing for 4 companies; these values are filled with the median inside the pipeline, and missing categorical values with the most frequent category.

3. **Feature engineering:** Region, PS_Ratio, the log10 variables, Cap_Tier and Sector_Group were created once, so later sections use the same definitions.
4. **Encoding:** Region and Sector_Group are converted to numbers with one-hot encoding (unknown categories are ignored at prediction time).
5. **Scaling:** numeric features are standardised with StandardScaler.
6. **Separation and split:** the input features (log_Revenue, log_Employees, Region, Sector_Group) are separated from the target (High_Cap). The data is split 80:20 into training and test sets with stratification, so both sets keep the same share of High-tier companies.
7. **No data leakage:** imputation, scaling and encoding are placed inside a scikit-learn Pipeline, so they are fitted on training data only.

3.3 Feature Extraction / Selection

No separate feature extraction technique was needed because the available company attributes are already meaningful. The four input features are log_Revenue and log_Employees (numeric, log-transformed because of skew), and Region and Sector_Group (categorical). Market cap itself is deliberately **not** used as a feature, because the target is derived from it and it would give the answer away. Features were selected using domain reasoning and checked with the correlation of each numeric feature with High_Cap; the correlation values are log_Revenue r = 0.51, log_Employees r = 0.19. Lasso regression, used in the regression practicals, additionally shows which features can be dropped.

3.4 Training Multiple Classification Models

Five classification models and one baseline were implemented, all inside the same preprocessing pipeline (Table 2).

Table 2: Classification models implemented

Sr. No.	Model Name	Purpose / Notes
0	Baseline (most frequent)	DummyClassifier that always predicts the majority class. A yardstick, not a candidate: a model that cannot beat it has learned nothing.
1	Logistic Regression	Linear model giving class probabilities; a simple, interpretable starting point (max_iter = 1000).
2	Decision Tree	Rule-based model with maximum depth 3, kept shallow because the training set is small.
3	Random Forest	Ensemble of 100 trees that reduces the variance of a single tree and captures non-linear patterns.
4	KNN (K = 5)	Predicts from the five most similar companies; works well after feature scaling.

Sr. No.	Model Name	Purpose / Notes
5	SVM (RBF kernel)	Finds a non-linear boundary between the classes using the radial basis function kernel.

3.5 Performance Evaluation and Model Comparison

The models were evaluated on the held-out test set (25 companies, 8 of them in the High tier) using accuracy, precision, recall, F1-score and the confusion matrix. Precision is the share of predicted High-tier companies that really are High tier; recall is the share of High-tier companies that were found; F1 balances the two. Because the test set is small, one company changes accuracy by several percentage points, so the ranking should be read as indicative rather than final. A majority-class baseline is included as a yardstick.

Table 3: Test-set performance of the classification models

Model	Accuracy	Precision	Recall	F1-Score
Baseline (most frequent)	0.6800	0.0000	0.0000	0.0000
Logistic Regression	0.8000	0.7143	0.6250	0.6667
Decision Tree	0.7600	0.6250	0.6250	0.6250
Random Forest	0.8400	0.7500	0.7500	0.7500
KNN (K=5)	0.7200	0.5714	0.5000	0.5333
SVM	0.7200	0.5714	0.5000	0.5333

3.6 Best Model Selection

The best model is the candidate with the highest test-set F1-score, with recall as the tie-breaker; the baseline is excluded from selection. F1 was chosen because, unlike a medical screening problem where a missed case is costly, no error type is much worse than the other here, so a balance of precision and recall is appropriate.

The selected model is **Random Forest**, with a test-set F1-score of 0.75 (baseline 0.00), an accuracy of 0.84 and a recall of 0.75. It is ahead of Logistic Regression (F1 0.67), the Decision Tree (0.63) and KNN and SVM (0.53 each), and all five classifiers beat the baseline, whose accuracy of 0.68 comes only from always predicting the majority class. Only 8 High-tier companies are in the test set, and the gap to Logistic Regression is one additional High-tier company found, so the choice should be treated as indicative. The model is saved as marketcap_classifier.joblib; the pipeline already contains the preprocessing, so no separate scaler file is needed.

3.7 Supporting Statistical Analysis

Alongside the classification task, the project applies the lab practicals to the same data. Market cap and revenue are heavily right-skewed, so t-tests, ANOVA and regression are run on the

log10 scale. Normality was checked with the Shapiro-Wilk test, and Welch's t-test is used for the US versus Non-US comparison because the groups differ in size (57 and 65 companies). The notebook does not apply a multiple-testing correction; this is listed as a limitation.

Table 4: Analyses carried out

Analysis	What was done
Descriptive statistics (Practical 5)	Mean, median, variance, standard deviation, skewness and IQR; t-based 95% confidence interval for the mean; one-sample t-test against a USD 500 billion benchmark on the raw and the log scale.
Probability distributions	Simulated Uniform, Normal / Log-normal, Binomial and Poisson distributions parameterised from the data; KS test against the uniform distribution, Shapiro-Wilk test on raw and log values, and a dispersion check for the Poisson assumption.
Sampling and CLT	Simple random, with replacement, systematic, stratified and cluster sampling compared with the population; Central Limit Theorem demonstrated with 1,000 sample means for n = 10, 30 and 100.
Two-sample and paired t-test (Practical 5-II)	Welch's t-test for US vs Non-US log market cap with a 95% confidence interval for the difference; paired t-test of log market cap vs log revenue.
ANOVA (Practical 5-II)	One-way ANOVA of log market cap across sectors with at least 5 companies; two-way ANOVA with Region and Sector for log market cap and log revenue.
Chi-square test (Practical 6)	Independence of Region and market-cap tier, with the expected counts checked.
Correlation (Practical 6 B&C)	Covariance, Pearson (raw and log10) and Spearman correlation between revenue and market cap, each with a significance test.
Regression (Practicals 7 and 8-B)	Simple linear regression on raw and log10 values with an OLS summary; Linear, Ridge, Lasso, Decision Tree and Random Forest regression of log market cap (Practicals 7 and 8-B) compared with a baseline.

Table 5: Summary of main hypothesis tests

Test	Null hypothesis (H0)	p-value	Decision at 0.05
One-sample t-test (log scale)	Geometric mean market cap = USD 500 B	$p = 2.7 \times 10^{-14}$	Reject
Welch t-test	US and Non-US mean log market cap are equal	$p = 7.7 \times 10^{-12}$	Reject
Paired t-test	Mean log market cap = mean log revenue	$p = 5.8 \times 10^{-31}$	Reject
One-way ANOVA	Mean log market cap is the same in every sector	$p = 0.837$	Fail to reject
Chi-square	Region and market-cap tier are independent	$p = 1.1 \times 10^{-9}$	Reject
Pearson (log10)	No correlation between log revenue and log market cap	$p = 5.8 \times 10^{-23}$	Reject
Spearman	No rank correlation between revenue and market cap	$p = 1.1 \times 10^{-18}$	Reject

Six of the seven tests are significant at the 5% level; only the one-way ANOVA across sectors is not. The notebook does not apply a multiple-testing correction, but the six significant p-values are all below 10^{-8}, far below the Bonferroni threshold for seven tests ($0.05 / 7 \approx 0.007$), so these conclusions would not change under such a correction.

4. PROJECT IMPLEMENTATION

4.1 Tools and Technologies

Table 6: Tools and technologies used

Tool / Technology	Version	Purpose
Python	version	Programming language
Google Colab	Hosted runtime	Development environment
Pandas / NumPy	versions	Data handling and numerical computing
yfinance	version	Collecting live company data from Yahoo Finance
SciPy / Statsmodels	versions	Hypothesis tests, distributions, OLS and ANOVA
Scikit-learn	version	Preprocessing pipelines, models and metrics
Matplotlib / Seaborn	versions	Visualization
Joblib	version	Saving and loading trained models
Streamlit	version	GUI framework for the dashboard
Gradio	version	GUI framework for the prediction interface

The versions can be printed in Colab with a cell such as import sys, pandas, sklearn; print(sys.version, pandas.__version__, sklearn.__version__).

4.2 Module-wise Implementation

- **Dataset loading and exploration module:** pulls data for the 123 tickers through the yfinance API, backs up the raw pull, converts to USD, runs the quality checks and plots the exploratory charts.
- **Data preprocessing module:** feature engineering, duplicate removal and a ColumnTransformer that imputes, scales and one-hot encodes inside a Pipeline.
- **Feature selection module:** defines the four input features and the High_Cap target, and excludes market cap to avoid leakage.
- **Model training and evaluation module:** trains the baseline and five classifiers, computes the metrics and confusion matrices, and builds the comparison table.
- **Best model saving and loading module:** selects the best candidate by test-set F1-score (recall as the tie-breaker), saves it with joblib, reloads it and verifies its accuracy.

- **GUI input and prediction module:** the function predict_cap_tier validates the input and returns the predicted tier with a probability; the Gradio app provides the web interface.

4.3 GUI Implementation

The interface is built with Gradio, which runs inside Google Colab and produces a web page with input fields and a result box. For the classification task, the prediction function predict_cap_tier(revenue_b, employees, region, sector) accepts four inputs: revenue in USD billion, number of employees, region (US or Non-US) and sector (one of the sectors the model knows; any other value is treated as "Other"). The project is also presented as a Streamlit dashboard titled "Global Listed Companies: Market Cap Dashboard", which shows the data source (Yahoo Finance via yfinance, all money values in USD billion), summary cards (number of companies, countries and sectors, median market cap and the largest company), charts such as companies by country and the US vs Non-US share of total market cap, and result cards for the best regression model and the best classifier, each shown against its baseline. The notebook also contains the Gradio app "Market Cap Prediction System", whose input fields are Revenue (USD billion) and Employees, pre-filled with the dataset medians, and whose output is a text box with the estimated market cap.

The prediction process is as follows: (1) the input is validated, and non-positive revenue or employee values and an invalid region are rejected with a message; (2) revenue and employees are converted to log10; (3) the values are placed in a one-row DataFrame with the same column names used in training; (4) the saved pipeline, loaded with joblib, applies imputation, scaling and encoding and then the best model; (5) the output is shown as "High market-cap tier" or "Not in the high tier", together with the probability of the high tier when the model provides probabilities.

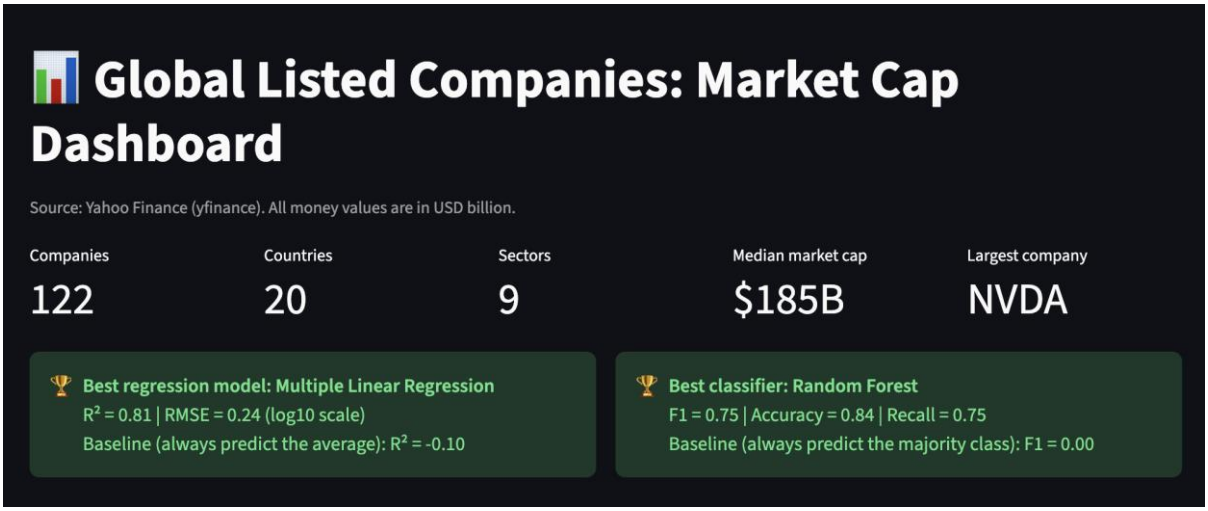


Figure 1: Streamlit dashboard: summary cards and best models

The dashboard header shows the data source, the dataset summary (122 companies, 20 countries, 9 sectors, median market cap $185B, largest company NVDA) and the best regression model and best classifier, each compared with its baseline.

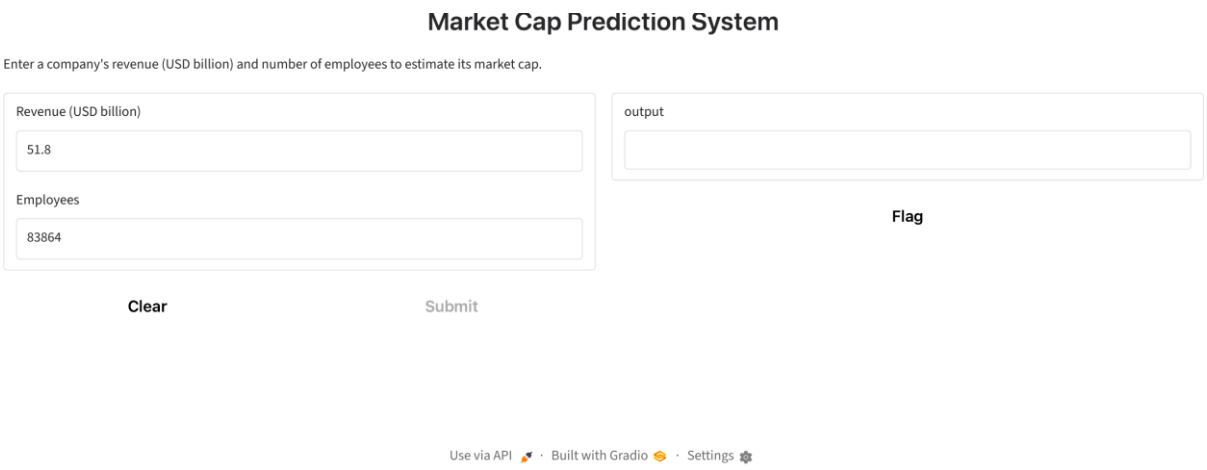


Figure 2: Gradio prediction interface

The Gradio interface of the notebook, with the input fields for revenue and employees pre-filled with the dataset medians and the output box for the estimated market cap.

5. COMPLETE CODE

The complete runnable source code is provided in the Google Colab notebook (market_cap_project_v7.ipynb), exported as PDF, and in the GitHub repository given in Section 8. The notebook is organised in four parts: Part A (data collection, cleaning, descriptive statistics, distributions, sampling, Central Limit Theorem, simple linear regression), Part B (Practicals 5, 5-II, 6, 6 B&C, 7, 8-B and 8-C), Part C (key findings and limitations) and Part D (the Streamlit dashboard). The code below reproduces the sections that produce the classification results and the prediction interface; it is taken from the notebook so that it corresponds to the reported results.

5.1 Setup: imports and settings

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import yfinance as yf
import joblib
from IPython.display import display

from scipy import stats
from scipy.stats import chi2 contingency, pearsonr, spearmanr, kstest, shapiro
import statsmodels.api as sm
from statsmodels.formula.api import ols

from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier, DummyRegressor
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Lasso, LinearRegression, LogisticRegression, Ridge
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
f1 score,
                             mean absolute error, mean squared error, precision score,
r2_score, recall_score)
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

np.random.seed(42)      # same random numbers on every run
alpha = 0.05            # significance level used in every hypothesis test
```

5.2 Data collection, cleaning and USD conversion

```
# Quick connectivity check (mirrors testing the endpoint before looping)
test = yf.Ticker("AAPL").info
print("Test ticker fetched:", test.get("longName"))
print("Sample field - marketCap:", test.get("marketCap"))

tickers = [
    # United States
    "AAPL", "MSFT", "GOOGL", "AMZN", "NVDA", "META", "TSLA", "BRK-B", "JPM", "V",
    "JNJ", "WMT", "PG", "XOM", "HD", "MA", "CVX", "KO", "PEP", "COST",
    "UNH", "BAC", "DIS", "ADBE", "CRM", "NFLX", "INTC", "CSCO", "PFE", "ABT",
    "ORCL", "AVGO", "LLY", "MRK", "ABBV", "MCD", "TMO", "QCOM", "AMD", "IBM",
    "GS", "MS", "WFC", "T", "VZ", "NKE", "LOW", "CAT", "GE", "HON",
    "UPS", "BA", "LMT", "AMGN", "MDT", "SBUX", "PM", "TXN", "LIN",
    # Taiwan / Japan / Korea
    "TSM", "UMC", "CHT", "SONY", "TM", "HMC", "MUFG", "SMFG", "KB",
    # Europe
    "SAP", "ASML", "NVO", "SHEL", "AZN", "NVS", "SIEGY", "UL", "BP", "GSK",
    "HSBC", "DEO", "SNY", "TTE", "ING", "BTI", "BUD", "ERIC", "NOK", "PHG", "RELX", "STM",
    # Canada
    "RY", "TD", "SHOP", "ENB", "CNQ", "BMO", "BNS", "CNI", "SU",
    # China
    "BABA", "TCEHY", "PDD", "JD", "BIDU", "NTES", "LI", "YUMC",
    # India (ADRs - more reliable via Yahoo than .NS tickers)
    "INFY", "HDB", "IBN", "WIT", "RDY",
    # Latin America, Australia, Israel
    "VALE", "PBR", "ITUB", "NU", "MELI", "BBD", "ABEV", "BHP", "TEVA", "CHKP", "NICE",
]
print("Tickers requested:", len(tickers))

records = []
for t in tickers:
    try:
        info = yf.Ticker(t).info
        market_cap = info.get("marketCap")
        revenue = info.get("totalRevenue")
        country = info.get("country")
        sector = info.get("sector")
        name = info.get("longName", t)
        employees = info.get("fullTimeEmployees")
        if market_cap and revenue:
            records.append({
                "Ticker": t,
                "Company": name,
                "Country": country if country else "Unknown",
                "Sector": sector if sector else "Unknown",
                "MarketCap": market_cap,
                "Revenue": revenue,
                "Employees": employees,
                "CapCurrency": info.get("currency") or "USD",
                "RevCurrency": info.get("financialCurrency") or "USD",
            })
    except Exception as e:
        print(f"Skipped {t}: {e}")
        continue
print("Total companies collected:", len(records))

df = pd.DataFrame(records)
assert len(df) >= 15, "Yahoo returned too little data. Wait a minute and run the previous cell
again."
df.to_csv("market_cap_raw.csv", index=False)      # backup of the raw pull

# Yahoo gives Revenue in the company's own currency and Market Cap in the trading currency.
# Convert both to USD, otherwise revenue cannot be compared between companies.
fallback rates = {"JPY": 150, "TWD": 31, "EUR": 0.92, "DKK": 6.9, "CNY": 7.2, "INR": 84,
"CAD": 1.38,
                  "GBP": 0.78, "CHF": 0.88, "KRW": 1400, "BRL": 5.5, "SEK": 10.5, "AUD": 1.5,
                  "ILS": 3.7, "MXN": 19, "HKD": 7.8, "NOK": 10.7}      # rough values, used
only if Yahoo fails
usd_rate = {"USD": 1.0}                                                 # local currency per 1
USD
for cur in set(df["CapCurrency"]) | set(df["RevCurrency"]):
    if cur != "USD":
        try:
            usd_rate[cur] = yf.Ticker(cur + "=X").history(period="5d")["Close"].iloc[-1]
        except Exception:
            usd_rate[cur] = fallback_rates[cur]
            print("Using a rough fallback exchange rate for", cur)
print("Exchange rates (local currency per 1 USD):", {c: round(float(r), 3) for c, r in
usd_rate.items()})

# USD billions
```

```
df["MarketCap_B"] = df["MarketCap"] / df["CapCurrency"].map(usd_rate) / 1e9
df["Revenue_B"] = df["Revenue"] / df["RevCurrency"].map(usd_rate) / 1e9

# Sanity check: after conversion no company here should have revenue above about US$1 trillion
print(df.nlargest(5, "Revenue B")[["Company", "Revenue B"]])
if df["Revenue_B"].max() > 1500:
    print("WARNING: revenue still looks like local currency. Check the exchange rates above.")
print("\nMissing values per column:")
print(df.isnull().sum())
print("Duplicate tickers:", df.duplicated("Ticker").sum())

df.to_csv("market_cap_data.csv", index=False)
```

5.3 Feature engineering

```
df["Region"] = np.where(df["Country"] == "United States", "US", "Non-US")
df["PS Ratio"] = df["MarketCap B"] / df["Revenue B"]
df["log MarketCap"] = np.log10(df["MarketCap B"])
df["log_Revenue"] = np.log10(df["Revenue_B"])
df["Employees"] = pd.to_numeric(df["Employees"], errors="coerce")
df["log_Employees"] = np.log10(df["Employees"])          # stays NaN where Employees is
missing
df["Cap_Tier"] = pd.qcut(df["MarketCap_B"], q=3, labels=["Low", "Medium", "High"])

sector_counts = df["Sector"].value_counts()
df["Sector_Group"] = df["Sector"].where(df["Sector"].map(sector_counts) >= 5, "Other")

print(df["Region"].value_counts())
print(df["Sector_Group"].value_counts())
```

5.4 Preprocessing pipeline

```
mc = df[["log_Revenue", "log_Employees", "Region", "Sector_Group", "log_MarketCap"]].copy()

# Identify Variables
numerical = mc.select_dtypes(include=np.number).columns.tolist()
categorical = mc.select_dtypes(exclude=np.number).columns.tolist()

# Remove duplicates (missing values are handled inside the pipeline, fitted on training data
only)
mc = mc.drop_duplicates().copy()

# Convert categorical variables to numbers (only for correlation and covariance analysis)
mc encoded = pd.get dummies(mc, columns=categorical, drop first=True, dtype=int)
print("Encoded Dataset:")
display(mc_encoded.head())

# Correlation analysis
correlation = mc_encoded.corr()
plt.figure(figsize=(12, 8))
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix")
plt.show()

# Covariance analysis
covariance = mc encoded.cov()
print("Covariance Matrix:")
display(covariance.round(3))
plt.figure(figsize=(12, 8))
sns.heatmap(covariance, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Covariance Matrix")
plt.show()

# Separate Features and Target
X = mc.drop("log_MarketCap", axis=1)
y = mc["log_MarketCap"]

# Identify Feature Types
num_features = X.select_dtypes(include=np.number).columns.tolist()
cat_features = X.select_dtypes(exclude=np.number).columns.tolist()

# Preprocessing Pipeline
preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), num_features),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
```

```
    ]), cat_features)
])

# Train-Test Split (80:20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

5.5 Classification data and train-test split

```
clf_data = df[["log_Revenue", "log_Employees", "Region", "Sector_Group"]].copy()
clf_data["High_Cap"] = (df["Cap_Tier"] == "High").astype(int)

print("Class counts (1 = High tier):")
print(clf data["High Cap"].value counts())

sns.countplot(x="High_Cap", data=clf_data)
plt.title("High market-cap tier (1) vs others (0)")
plt.show()

print("\nCorrelation of numeric features with High_Cap:")
print(clf_data[["log_Revenue", "log_Employees",
"High_Cap"]].corr()["High_Cap"].drop("High_Cap").round(3))

Xc = clf_data.drop(columns="High_Cap")
yc = clf_data["High_Cap"]

# 80-20 split; stratify keeps the same share of High-tier companies in train and test
Xc_train, Xc_test, yc_train, yc_test = train_test_split(Xc, yc, test_size=0.2,
random_state=42, stratify=yc)
print("Training data:", Xc_train.shape, "| Testing data:", Xc_test.shape)
print("High-tier companies in test set:", int(yc_test.sum()), "of", len(yc_test))
```

5.6 Training and evaluating the classification models

```
clf_models = {
    "Baseline (most frequent)": DummyClassifier(strategy="most_frequent"),
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree": DecisionTreeClassifier(max_depth=3, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "KNN (K=5)": KNeighborsClassifier(n neighbors=5),
    "SVM": SVC(kernel="rbf", random state=42)
}

clf pipelines = {}
clf rows = []
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.ravel()

for ax, (name, clf) in zip(axes, clf_models.items()):
    pipe = Pipeline([
        ("preprocessing", clone(preprocessor)),
        ("model", clf)
    ])
    pipe.fit(Xc_train, yc_train)
    pred = pipe.predict(Xc_test)
    clf_pipelines[name] = pipe

    print(f"--- Metrics for {name} on Test Set ---")
    print(f"Accuracy : {accuracy_score(yc_test, pred):.4f}")
    print(f"Precision: {precision_score(yc_test, pred, zero_division=0):.4f}")
    print(f"Recall   : {recall score(yc test, pred, zero division=0):.4f}")
    print(f"F1-Score : {f1 score(yc test, pred, zero division=0):.4f}")
    print(classification report(yc test, pred, zero division=0))

    sns.heatmap(confusion_matrix(yc_test, pred, labels=[0, 1]), annot=True, fmt="d",
cmap="Blues", ax=ax)
    ax.set_title(f"{name} - Confusion Matrix")
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

    clf_rows.append({
        "Model": name,
        "Accuracy": accuracy_score(yc_test, pred),
        "Precision": precision_score(yc_test, pred, zero_division=0),
        "Recall": recall score(yc test, pred, zero division=0),
        "F1-Score": f1 score(yc test, pred, zero division=0)
    })

plt.tight layout()
plt.show()
```

5.7 Model comparison and best model selection

```
comparison_df = pd.DataFrame(clf_rows)
print("--- Model Comparison Table ---")
display(comparison df.round(4))

comparison_df.set_index("Model").plot(kind="bar", figsize=(12, 5), ylim=(0, 1.05))
plt.title("Model Comparison")
plt.xticks(rotation=0)
plt.show()

# Best model = highest F1-Score (the baseline is only a yardstick, so it is not a candidate)
candidates = comparison_df[comparison_df["Model"] != "Baseline (most frequent)"]
best_row = candidates.sort_values(by=["F1-Score", "Recall"], ascending=False).iloc[0]
best_name = best_row["Model"]
best_clf = clf_pipelines[best_name]
print("Best model:", best_name)
```

5.8 Saving the best model and the prediction function

```
# Save the best model (the pipeline already contains the preprocessing, so no separate scaler
file is needed)
joblib.dump(best_clf, "marketcap_classifier.joblib")
print("Saved: marketcap_classifier.joblib")

# Check that the saved model loads and works
loaded_clf = joblib.load("marketcap_classifier.joblib")
y_check = loaded_clf.predict(Xc_test)
print("Accuracy of loaded model:", accuracy_score(yc_test, y_check))

classifier = joblib.load("marketcap classifier.joblib")
known_sectors = sorted(df["Sector_Group"].unique())
print("Sectors the model knows:", known_sectors)

def predict cap tier(revenue b, employees, region, sector):
    if revenue b <= 0 or employees <= 0:
        return "Enter positive values for revenue and employees."
    if region not in ("US", "Non-US"):
        return "Region must be 'US' or 'Non-US'."
    sector = sector if sector in known_sectors else "Other"
    row = pd.DataFrame([{
        "log_Revenue": np.log10(revenue_b),
        "log_Employees": np.log10(employees),
        "Region": region,
        "Sector_Group": sector,
    }])
    label = "High market-cap tier" if classifier.predict(row)[0] == 1 else "Not in the high
tier"
    if hasattr(classifier, "predict proba"):
        label += f" (probability of high tier: {classifier.predict_proba(row)[0][1]:.0%})"
    return label

example_sector = known_sectors[0]
print(predict_cap_tier(400, 150000, "US", example_sector))
print(predict_cap_tier(20, 30000, "Non-US", example_sector))
```

5.9 Market cap regression (Practical 7) and Gradio interface

```
# Feature Selection
X = df[["log Revenue", "log Employees"]]
X = X.fillna(X.median())          # one or two companies have no employee count
y = df["log MarketCap"]

# Splitting into Training and Testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardisation (fit on train only)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

linear_model = LinearRegression()
linear model.fit(X train scaled, y train)
ridge model = Ridge(alpha=1.0)
ridge model.fit(X train scaled, y train)
lasso_model = Lasso(alpha=0.05)
lasso_model.fit(X_train_scaled, y_train)
tree model = DecisionTreeRegressor(max depth=3, random state=42)
tree model.fit(X train scaled, y train)
forest_model = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
forest_model.fit(X_train_scaled, y_train)

# Baseline: always predicts the average market cap
baseline model = DummyRegressor(strategy="mean")
baseline model.fit(X train scaled, y train)

models = {
    "Baseline (predict the mean)": baseline model,
    "Linear Regression": linear model,
    "Ridge Regression": ridge model,
    "Lasso Regression": lasso_model,
    "Decision Tree Regression": tree_model,
    "Random Forest Regression": forest_model,
}

result = []
for name, m in models.items():
    pred = m.predict(X_test_scaled)
    result.append({
        "Model": name,
        "MSE": mean squared error(y test, pred),
        "MAE": mean absolute error(y test, pred),
        "R2": r2_score(y_test, pred),
    })

result_df = pd.DataFrame(result).sort_values("R2", ascending=False).reset_index(drop=True)
print(result_df.round(3))

# The baseline is a yardstick, so the best model is chosen from the other rows
best_row = result_df[result_df["Model"] != "Baseline (predict the mean)"].iloc[0]
best_model_name = best_row["Model"]
best_model = models[best_model_name]

joblib.dump(best model, "best model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(list(X.columns), "feature names.pkl")

model = joblib.load("best_model.pkl")
scaler = joblib.load("scaler.pkl")
features = joblib.load("feature names.pkl")

def predict_market_cap(revenue_b, employees):
    # takes RAW inputs (revenue in USD billion, number of employees) and takes the log inside
    row = pd.DataFrame([[np.log10(revenue_b), np.log10(employees)]], columns=features)
    result = model.predict(scaler.transform(row))[0]
    return f"Predicted market cap: about USD {10 ** result:,.0f} billion"

# Web interface (needs gradio; in Colab it is available, elsewhere: pip install gradio)
try:
    import gradio as gr
    inputs = [
        gr.Number(label="Revenue (USD billion)", value=round(float(df["Revenue B"].median()),
1)),
        gr.Number(label="Employees", value=int(df["Employees"].median())),
    ]
    interface = gr.Interface(
        fn=predict_market_cap,
        inputs=inputs,
        outputs="text",
        title="Market Cap Prediction System",
        description="Enter a company's revenue (USD billion) and number of employees to
estimate its market cap.",
    )
    interface.launch()
except ImportError:
    print("gradio is not installed, skipping the web interface.")
```

6. RESULTS AND DISCUSSION

This section presents the outputs of the implemented project. Each figure is a screenshot from the student's own run of the notebook.

6.1 Required Output Screenshots

	Ticker	Company	Country	Sector	MarketCap	Revenue	Employees	CapCurrency	RevCurrency	MarketCap_B	Revenue_B
0	AAPL	Apple Inc.	United States	Technology	5017916932096	466822987776	150000.000000	USD	USD	5017.916932	466.822988
1	MSFT	Microsoft Corporation	United States	Technology	3943039238144	331839012864	223000.000000	USD	USD	3943.039238	331.839013
2	GOOGL	Alphabet Inc.	United States	Communication Services	3973383716864	445865984000	198933.000000	USD	USD	3973.383717	445.865984
3	AMZN	Amazon.com, Inc.	United States	Consumer Cyclical	2746195378176	775680032768	1595000.000000	USD	USD	2746.195378	775.680033
4	NVDA	NVIDIA Corporation	United States	Technology	5200780722176	302970011648	42000.000000	USD	USD	5200.780722	302.970012
5	META	Meta Platforms, Inc.	United States	Communication Services	1594209206272	241605001216	66775.000000	USD	USD	1594.209206	241.605001
6	TSLA	Tesla, Inc.	United States	Consumer Cyclical	1597772529664	103569997824	134785.000000	USD	USD	1597.772530	103.569998
7	BRK-B	Berkshire Hathaway Inc.	United States	Financial Services	1156711186432	384687013888	387800.000000	USD	USD	1156.711186	384.687014
8	JPM	JPMorgan Chase & Co.	United States	Financial Services	901045092352	190732992512	322964.000000	USD	USD	901.045092	190.732993
9	V	Visa Inc.	United States	Financial Services	752068526080	44487999488	34100.000000	USD	USD	752.068526	44.487999

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 122 entries, 0 to 121
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Ticker       122 non-null    object
 1   Company      122 non-null    object
 2   Country      122 non-null    object
 3   Sector       122 non-null    object
 4   MarketCap    122 non-null    int64
 5   Revenue      122 non-null    int64
 6   Employees    118 non-null    float64
 7   CapCurrency  122 non-null    object
 8   RevCurrency  122 non-null    object
 9   MarketCap_B  122 non-null    float64
 10  Revenue_B    122 non-null    float64
dtypes: float64(3), int64(2), object(6)
memory usage: 10.6+ KB
```

```
df.shape
```

```
(122, 11)
```

Figure 3: Dataset loading and exploration output

The first rows and the structure of the cleaned dataset, confirming the columns, data types and number of companies.

```
                          Company   Revenue_B
3                 Amazon.com, Inc.  775.680033
11                    Walmart Inc.  735.839977
0                       Apple Inc.  466.822988
20  UnitedHealth Group Incorporated  450.059010
2                     Alphabet Inc.  445.865984

Missing values per column:
Ticker         0
Company        0
Country        0
Sector         0
MarketCap      0
Revenue        0
Employees      4
CapCurrency    0
RevCurrency    0
MarketCap_B    0
Revenue_B      0
dtype: int64
Duplicate tickers: 0

Class counts (1 = High tier):
High_Cap
0    81
1    41
Name: count, dtype: int64
```

```
Training data: (97, 4) | Testing data: (25, 4)
High-tier companies in test set: 8 of 25
```

Figure 4: Data preprocessing output

Results of the quality checks and the stratified 80:20 split. The class counts show that the High tier is about one third of the data.

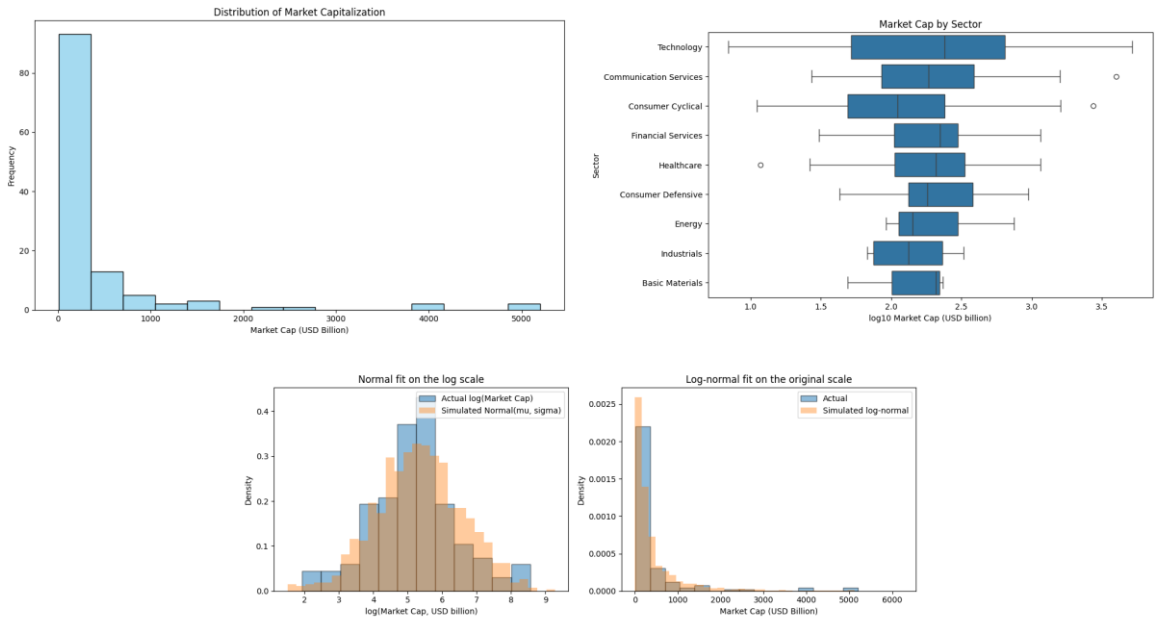


Figure 5: Distribution of market capitalization

The raw market cap is strongly right-skewed (skewness 3.93), while the log scale is far more symmetric (skewness 0.22), which is why the log scale is used for the tests and models. The box plot compares log10 market cap across sectors.

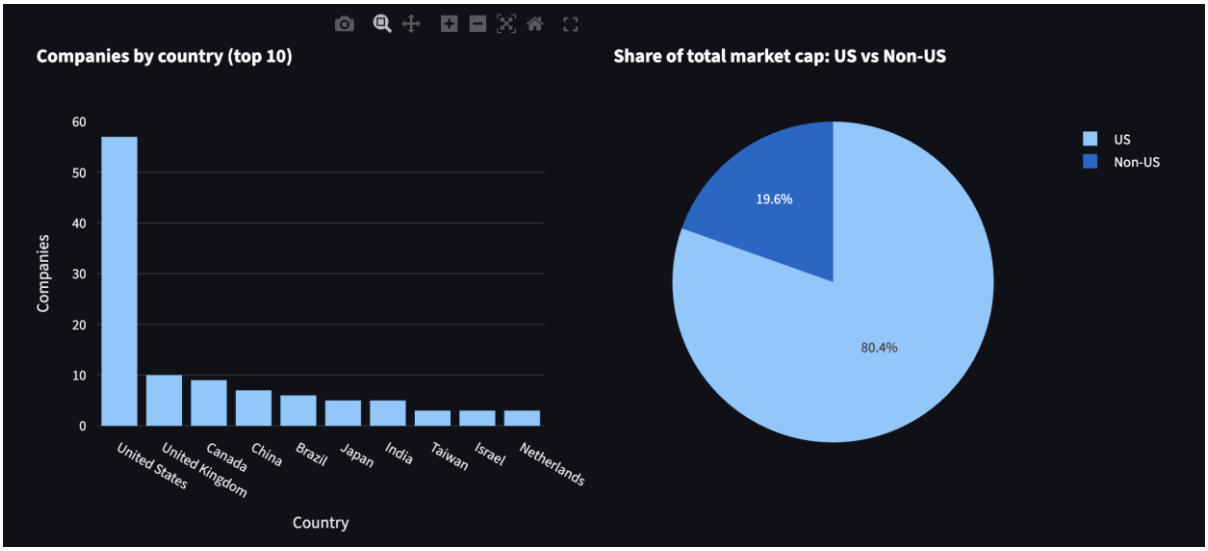


Figure 6: Companies by country and US vs Non-US share of market cap

The bar chart shows that the sample is dominated by US companies, followed by the United Kingdom, Canada and China. The pie chart shows that US companies hold 80.4% of the total market cap of the sample and Non-US companies 19.6%.

```
--- Model Comparison Table ---
```

	Model	Accuracy	Precision	Recall	F1-Score
0	Baseline (most frequent)	0.68	0.0000	0.000	0.0000
1	Logistic Regression	0.80	0.7143	0.625	0.6667
2	Decision Tree	0.76	0.6250	0.625	0.6250
3	Random Forest	0.84	0.7500	0.750	0.7500
4	KNN (K=5)	0.72	0.5714	0.500	0.5333
5	SVM	0.72	0.5714	0.500	0.5333

Figure 7: Model comparison table

Test-set accuracy, precision, recall and F1-score for all classification models and the baseline.

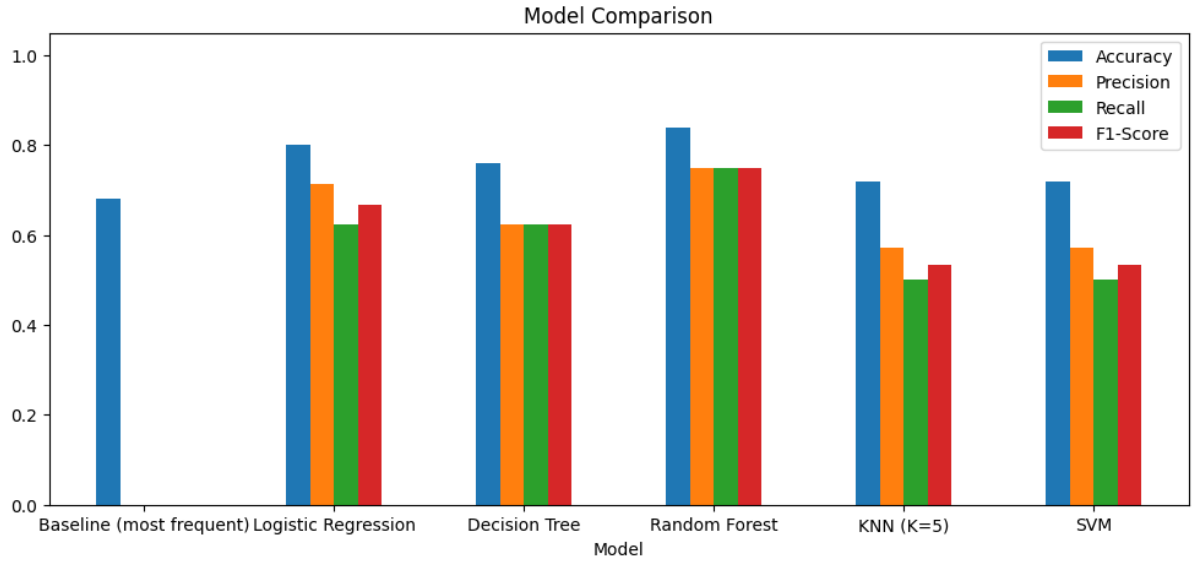


Figure 8: Accuracy and performance results

A bar chart comparing the metrics of the models on the test set.

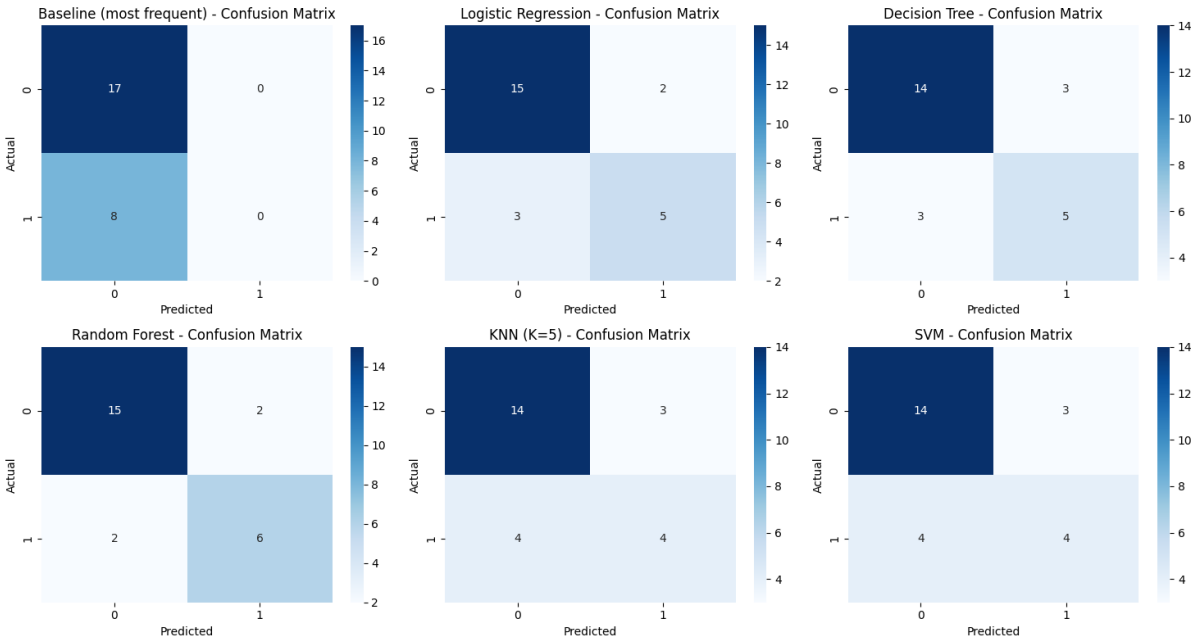


Figure 9: Confusion matrices

Each matrix shows how many High-tier and other companies each model classified correctly and incorrectly on the test set.

```
Best model: Random Forest

Saved: marketcap_classifier.joblib
Accuracy of loaded model: 0.84
```

Figure 10: Best model selection

The model with the highest test-set F1-score is selected (the baseline is excluded); the saved model reproduces the test accuracy of 0.84 when it is loaded again.

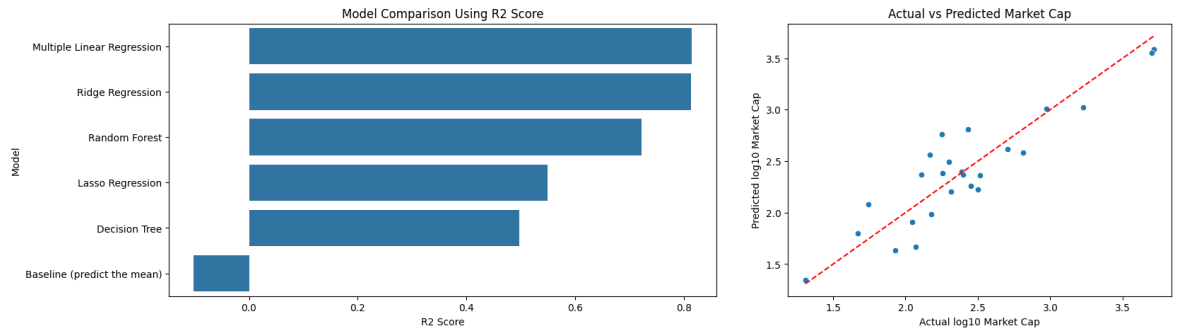


Figure 11: Regression model comparison

Test-set R^2 of the regression models against the baseline (Practical 8-B), and the actual versus predicted market cap on the log10 scale for the best model, Multiple Linear Regression.

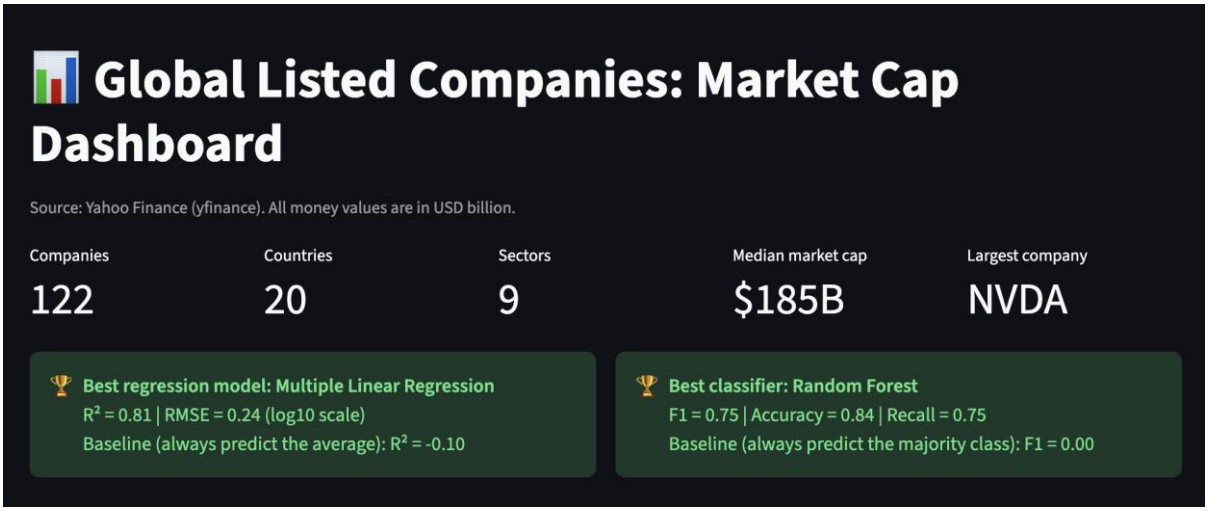


Figure 12: GUI interface (dashboard)

The Streamlit dashboard interface of the project, with the dataset summary and the best-model result cards.

```
print(predict_cap_tier(400, 150000, "US", example_sector))
print(predict_cap_tier(20, 30000, "Non-US", example_sector))

High market-cap tier (probability of high tier: 77%)
Not in the high tier (probability of high tier: 0%)
```

Figure 13: GUI prediction results

Predictions of the saved classifier (sector: Communication Services): a US company with USD 400 billion revenue and 150,000 employees is placed in the high tier (probability 77%), while a Non-US company with USD 20 billion revenue and 30,000 employees is not (probability 0%).

6.2 Result Analysis

Model performance. Among the five classifiers, Random Forest was selected as the best model. It obtained an F1-score of 0.75, an accuracy of 0.84, a precision of 0.75 and a recall of 0.75, compared with an F1-score of 0.00 for the majority-class baseline, which never finds a High-tier company. Logistic Regression was the closest competitor (accuracy 0.80, F1 0.67), followed by the Decision Tree (accuracy 0.76, F1 0.63); KNN and SVM were clearly weaker, with identical scores (accuracy 0.72, F1 0.53) that are still above the baseline's F1.

Errors. On the test set the Random Forest found 6 of the 8 High-tier companies (6 true positives) and correctly rejected 15 of the 17 others (15 true negatives), with 2 false positives and 2 false negatives, so both error types are equally frequent. The baseline classified all 25 companies as not High tier (17 true negatives, 8 false negatives). The notebook does not list which companies were misclassified; because the High tier is defined by a cut-off on market cap, errors are most plausibly companies close to that cut-off, but this was not checked.

Statistical findings. On the log scale, the typical US company is 4.26 times the size of a Non-US company (Welch t-test $p = 7.7 \times 10^{-12}$). The sector effect on market cap is not significant (one-way ANOVA $p = 0.837$), and in the two-way ANOVA only Region is significant ($p = 1.1 \times 10^{-7}$) while Sector ($p = 0.58$) and the interaction ($p = 0.58$) are not. Region and market-cap

tier are associated (chi-square $p = 1.1 \times 10^{-9}$): 34 of the 57 US companies are in the High tier against 7 of the 65 Non-US companies. The log-log correlation between revenue and market cap is $r = 0.75$, and a 1% rise in revenue goes with about 0.87% higher market cap, with revenue alone explaining 56% of the variation ($R^2 = 0.557$). For regression, the best model (Multiple Linear Regression) has an R^2 of 0.81 and an RMSE of 0.24 on the log10 scale, against an R^2 of −0.10 for the baseline that always predicts the average. In this sample, US companies hold 80.4% of the total market cap.

Strengths. Revenue and employees give the models real, if partial, information about size; preprocessing inside a pipeline prevents data leakage; and the baseline shows honestly whether a model has learned anything.

Limitations. The test set is small, so one company moves the accuracy by several percentage points. The sample is hand-picked and biased towards large, mostly US-listed companies, and companies that shrank or disappeared are missing (survivorship bias). Market value mostly reflects profit, growth and risk, which are not among the inputs, so only moderate predictive power is expected. Several groups have only a few companies, and with many tests running at once some "significant" results can arise by chance, because the notebook applies no multiple-testing correction. All findings show association, not causation.

Objectives achieved. The data was collected, converted to USD and quality-checked; distributions, sampling and the Central Limit Theorem were demonstrated; hypotheses about size, region and sector were tested; several regression and classification models were trained and compared with a baseline; and the best model was saved and made available through a prediction interface.

7. CONCLUSION

This project studied what is related to the market capitalization of the world's largest listed companies using live data from Yahoo Finance for 122 companies from 20 countries and 9 sectors. After converting everything to USD and moving to the log10 scale to deal with extreme skewness, the project applied descriptive statistics, distribution fitting, sampling techniques, the Central Limit Theorem, t-tests, ANOVA, chi-square and correlation tests, and built regression and classification models.

For the classification task, five models (Logistic Regression, Decision Tree, Random Forest, KNN and SVM) were compared with a majority-class baseline using accuracy, precision, recall, F1-score and confusion matrices. The final selected model is Random Forest, with a test-set F1 of 0.75, accuracy of 0.84 and recall of 0.75 (a majority-class baseline scores an F1 of 0.00), and it is used in the prediction interface. Revenue and region were clearly related to market capitalization (log-log correlation 0.75; a typical US company is about 4.3 times the size of a Non-US one), while sector showed no significant difference in size (ANOVA $p = 0.84$).

The main limitations are the hand-picked and small sample, the single-date snapshot, the small number of predictors and the small size of some groups. Possible improvements are to add profitability (net margin, return on equity), growth and valuation ratios (P/E) as features, to use

several snapshots over time (panel data), to draw companies systematically from an index such as MSCI World to remove selection bias, and to try gradient-boosted models once there are more companies and features.

8. GITHUB LINK

GitHub Repository URL: https://github.com/AayushK401/global-market-cap-analysis.git

The repository contains the Python code files, the dataset (market_cap_data.csv and the cached market_cap_raw.csv), the trained model files (marketcap_classifier.joblib, marketcap_regression_pipeline.joblib, best_model.pkl, scaler.pkl, feature_names.pkl), the GUI application code and the Colab notebook.

9. REFERENCES

[1]Yahoo Finance, market data for listed companies. https://finance.yahoo.com/ (accessed via the yfinance library).

[2]yfinance: Yahoo! Finance market data downloader. https://pypi.org/project/yfinance/

[3]Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python”, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. https://scikit-learn.org/stable/

[4]pandas documentation. https://pandas.pydata.org/docs/

[5]SciPy documentation (scipy.stats). https://docs.scipy.org/doc/scipy/

[6]Statsmodels documentation. https://www.statsmodels.org/stable/

[7]Matplotlib documentation. https://matplotlib.org/stable/ and Seaborn documentation. https://seaborn.pydata.org/

[8]Joblib documentation. https://joblib.readthedocs.io/

[9]Gradio documentation. https://www.gradio.app/docs